KIRA AI Incident Report

Cluster Summary

The cluster is a single-node `minikube` development/testing environment running microservices. Currently, the cluster is in a **Recovered/Degraded** state.

* **Deployments:** 7 out of 7 deployments are reporting healthy status (`1/1` available replicas).
* **Workloads:** 8 pods are running, but **6 out of these 8 pods** (`auth`, `boutique-postgres-0`, `frontend`, `order-service`, `product-service`, and `user-service`) have restarted exactly **1 time**.
* **Current State:** All pods are currently in a `Running` state. However, the synchronized restarts across multiple unrelated services indicate a systemic node-level or control-plane event.

Key Findings

1. **Systemic Restart Pattern:** The restarts are not isolated to a single microservice. Six structurally independent components (including a StatefulSet database `boutique-postgres-0` and stateless Java/Node deployments) restarted concurrently.
2. **Selective Survival:** Only the `gateway` and `orders` pods have `0` restarts.
3. **Low CPU Footprint:** All pods exhibit extremely low CPU usage. Notably, the `frontend` pod is showing almost zero CPU activity (`0.000012`), suggesting it has recently initialized and is not yet serving active traffic, or is experiencing post-restart cold-start latency.
4. **Lack of Log Warnings:** There are no active logs captured in Loki, indicating the restart happened abruptly, preventing the application runtimes from flushing graceful shutdown logs, or the logging agent itself was interrupted.

Severity Assessment

Severity: **SEV-2 (Moderate / Systemic Degradation)**

While the applications have successfully restarted and returned to a `Running` state with `1/1` replicas, a simultaneous restart across stateful and stateless workloads on a single node points to host-level instability. If this occurs in production, it represents a brief but total outage of the system.

Root Cause Analysis

Based on the correlated metric signatures, the root cause is a **Host-Level Event** or **Container Runtime Interruption** on the `minikube` node, rather than individual application bugs (such as Out-Of-Memory/OOM kills or NullPointerExceptions).

Probable Hypotheses:
1. **Minikube Pause/Resume or Host Sleep:** Because this is running on `minikube`, a host laptop/server going to sleep or hibernation suspends the hypervisor/container runtime. When resumed, Kubernetes registers this as a transition, often causing the Kubelet to restart container runtimes, resulting in a single restart across all running pods.
2. **Kubelet/Docker Daemon Restart:** A restart of the Docker/Containerd service on the `minikube` node would terminate all container processes. Once the daemon recovered, Kubelet restarted the containers, incrementing the restart count to `1`.
3. **Resource Starvation (Node-level):** If the host machine ran out of memory, the OS kernel's OOM Killer may have terminated the Docker daemon or Kubelet itself, leading to a cascade restart of workloads once system services recovered.

Recommended Remediation

To restore high confidence and prevent unexpected restarts:

1. **Inspect Termination Causes:** Retrieve the `ExitCode` and `Reason` of the terminated containers using the `lastState` field to confirm if they were terminated by an external signal (like `SIGKILL` or `SIGTERM`).
2. **Correlate Node Events:** Check system-level Kubernetes events to determine if a Node `Rebooted` or if `Kubelet` lost connection.
3. **Verify App Health:** Ensure the `frontend` pod is indeed ready to receive traffic and its low CPU usage is not due to a hung startup probe.

Preventative Measures

1. **Set Resource Requests & Limits:** Implement explicit CPU and Memory requests/limits on all deployments to prevent node-level resource exhaustion from triggering host-level OOM events.
2. **Liveness & Readiness Probes:** Ensure all deployments have configured `livenessProbes` and `readinessProbes` to gracefully handle runtime interruptions and prevent traffic from hitting uninitialized pods (like `frontend`).
3. **Multi-Node Topologies (Production):** Transition workloads away from single-node instances (`minikube`) to multi-node clusters with Pod Anti-Affinity rules to ensure single-node reboots do not cause total service disruption.

Exact kubectl Commands to diagnose and fix the issue

Execute the following commands to diagnose the exact reason behind the restarts:

1. Inspect Last State and Exit Codes of restarted pods
```bash
kubectl get pods -n default -o jsonpath='{range .items[*]}{.metadata.name}{"\tLast State: "}{.status.containerStatuses[*].lastState}{"\n"}{end}'
```
Look specifically for `OOMKilled` or `ExitCode: 137` (SIGKILL) / `143` (SIGTERM).

2. Check Node-level events and Kubelet status
```bash
kubectl describe node minikube | grep -A 10 Conditions
```
```bash
kubectl get events --sort-by='.metadata.creationTimestamp' -n default
```
Look for `NodeNotReady`, `SystemOOM`, or `Rebooted` events.

3. Check the logs of the previous container instance for the frontend pod
```bash
kubectl logs frontend-75bc59b865-td9rw --previous --tail=100
```

Suggested kubectl Commands

```bash
# Get detailed status of all pods including restart reasons
kubectl get pods -o wide

# Check if there are any pending restarts or crashing pods
kubectl get events --field-selector type=Warning

# Check resource usage on the minikube node
```

kubectl top node minikube
```
```